

Web Programming Lab

JavaScript Objects & Functions

Week 7



Hello, World! (Again)

```
<!-- index.html -->

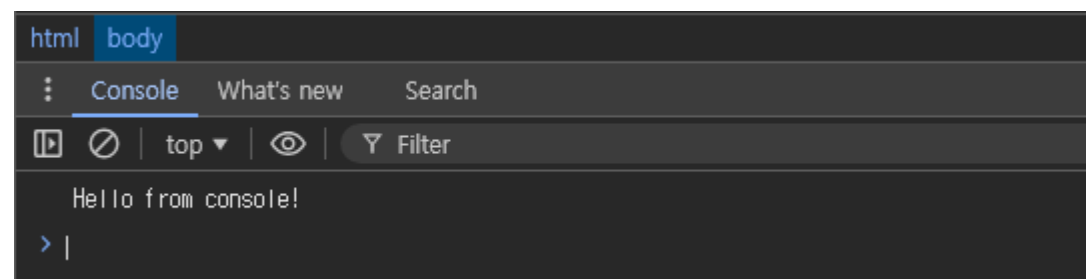
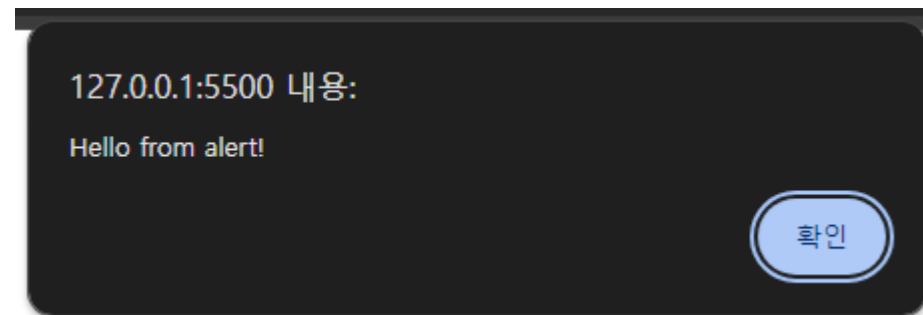
<!DOCTYPE html>

<body>
  <h1>Nothing here</h1>
</body>
<script src="./script.js"></script>
```

```
// script.js

"use strict";

alert("Hello from alert!");
console.log("Hello from console!");
```



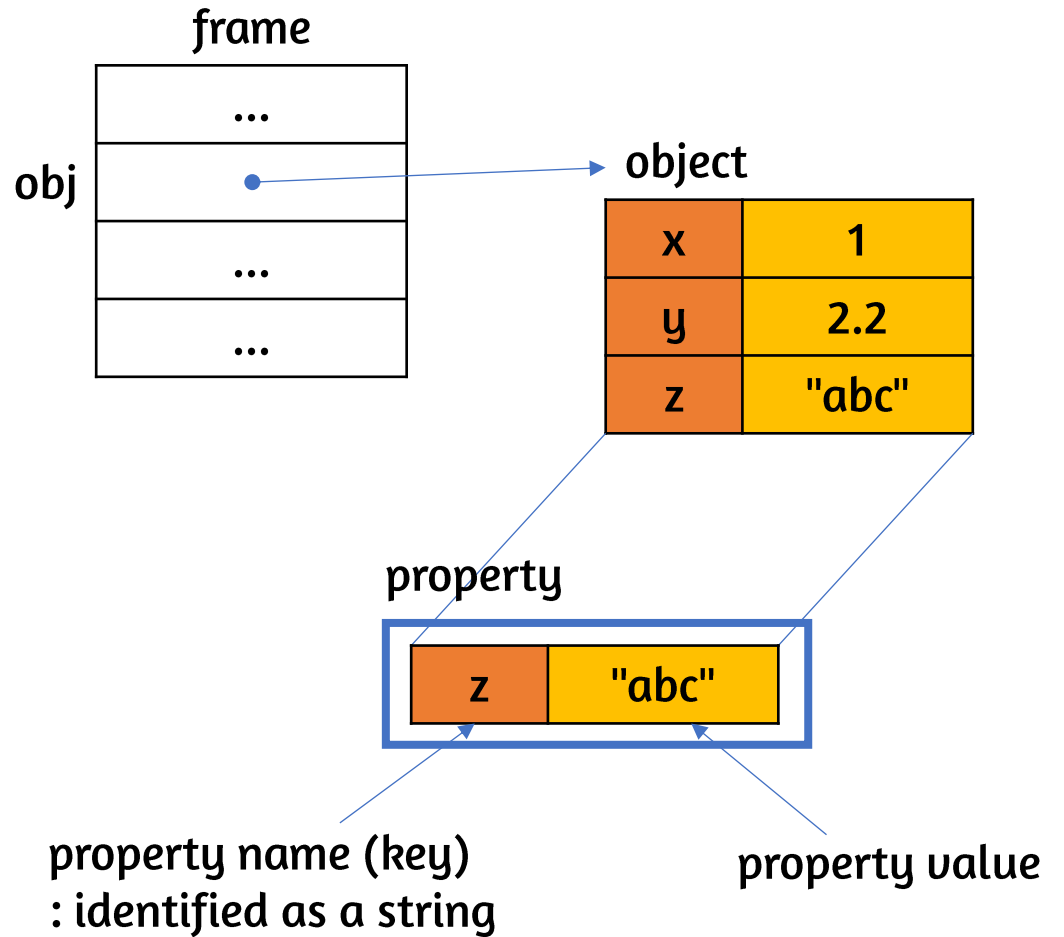
Object

```
// script.js
"use strict";
let obj = { x: 1, y: 2.2, z: "abc" };

// How about this?
let obj = { 0: 123, 2.2: 456 };

console.log(obj);
```

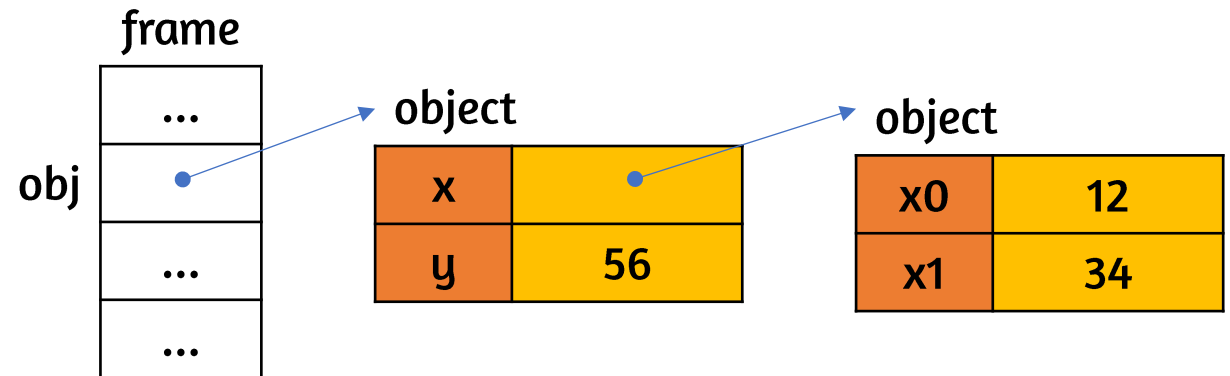
```
▶ {x: 1, y: 2.2, z: 'abc'}
>
```



Object in Object

```
// script.js  
"use strict";  
let obj = { x: { x0: 12, x1: 34 }, y: 56 };  
console.log(obj);
```

```
▼ {x: {...}, y: 56} i  
  ► x: {x0: 12, x1: 34}  
    y: 56  
  ► [[Prototype]]: Object
```





Accessing Property

```
"use strict";

let book = {
  author: "Enrico",
  title: "Learning JS",
  for: "students",
  pages: 340,
  "chapter pages": [90, 50, 60, 140]
};
let authorname = book.author;
let booktitle = book["title"];

// 1
let chapter2 = book["chapter pages"][2];
chapter2 = 100;
console.log(book["chapter pages"]);

// -> continued in next box
```

```
// -> continued from previous box

// 2
book["chapter pages"][3] = 200;
console.log(book["chapter pages"]);

// 3
let pages = "chapter pages";
console.log(book.pages);
console.log(book[pages]);
console.log(book["pages"]);
```

Adding/Deleting Property

```
"use strict";

let book = {
  author: "Enrico",
  title: "Learning JS",
  for: "students",
  pages: 340,
  "chapter pages": [90, 50, 60, 140]
};

// adding property
book.published_year = 2010;

// deleting property
delete book["chapter pages"];

console.log(book);
```

```
script.js:17
{author: 'Enrico', title: 'Learning JS', for:
▼ 'students', pages: 340, published_year: 2010}
  i
    author: "Enrico"
    for: "students"
    pages: 340
    published_year: 2010
    title: "Learning JS"
    ► [[Prototype]]: Object
```

>



Property & Error

```
"use strict";
let book = {
  author: { name: "Enrico", age: 40 },
  title: "Learning JS",
};

let authorname = book.author.name; // 1: not that good

// 2: better
let authorname = undefined;
if (book) {
  if (book.author) {
    authorname = book.author.name;
  }
}

// 3: better
let authorname = book && book.author && book.author.name;
```



Iterate Over Property

```
"use strict";

let book = {
  author: "Enrico",
  pages: 340,
  "chapter pages": [90, 50, 60, 140]
};

let k = Object.keys(book);
console.log(k);

let kv = Object.entries(book);
console.log(kv);
```

```
▶ (3) ['author', 'pages', 'chapter pages']
▼ (3) [Array(2), Array(2), Array(2)] i
  ▶ 0: (2) ['author', 'Enrico']
  ▶ 1: (2) ['pages', 340]
  ▶ 2: (2) ['chapter pages', Array(4)]
    length: 3
  ▶ [[Prototype]]: Array(0)
```

>



Iterate Over Property

```
"use strict";

let book = {
  author: "Enrico",
  pages: 340,
  "chapter pages": [90, 50, 60, 140]
};

for (let aa in book) {
  console.log("aa: " + aa);
}

for (let bb in Object.keys(book)) {
  console.log("bb: " + bb);
}

for (let cc of Object.entries(book)) {
  console.log("cc: " + cc);
}
```

```
aa: author
aa: pages
aa: chapter pages
bb: 0
bb: 1
bb: 2
cc: author, Enrico
cc: pages, 340
cc: chapter pages, 90, 50, 60, 140
```

>



Object.assign

```
"use strict";

let obj = { x: 11, y: [22, 33] };

let obj2 = Object.assign({ x: 44, z: 55 }, obj);
// it acts like:
// let tmp = { x: 44, z: 55 };
// for (let keys in obj) {
//     tmp[keys] = obj[keys];
// }
// let obj2 = tmp;

console.log(obj2);
```

```
▼ {x: 11, z: 55, y: Array(2)} ⓘ
  x: 11
  ► y: (2) [22, 33]
  z: 55
  ► [[Prototype]]: Object
>
```



Copying Object

```
"use strict";

let obj = { x: 11, y: [ 22, 33 ] };

let copy1 = obj;

// shallow copy
let copy2 = Object.assign({}, copy1);

// deep copy
let copy3 = JSON.parse(JSON.stringify(obj));
```

- Frame/object layout?
- What if copy2.y.push(44)?



Creating Object Using Function

```
"use strict";

function Employee(name, salary, role = "TBD") {
    this.name = name;
    this.salary = salary;
    this.role = role;
    return this;
}

let emp1 = new Employee("Kim", 100, "Backend Developer");
let emp2 = new Employee("Lee", 150, "Project Manager");
let emp3 = new Employee("Park", 70);
```



Function

```
"use strict";

function fn1(x) {
    return x * 2;
}

let fn2 = function func2(x) {
    return x * 2;
};

let fn3 = function(x) {
    return x * 2;
};

let fn4 = (x) => {
    return x * 2;
};
```

- Call fn1(), fn2(), fn3(), fn4()
- Can you call func2()?
- How fn2 and fn3 different?



Closure

```
"use strict";

function greeter(name) {
  const myname = name;
  const hello = function() {
    return "Hello " + myname;
  };
  return hello;
}

const helloTom = greeter("Tom");
const helloJerry = greeter("Jerry");
console.log(helloTom());
console.log(helloJerry());
```

- myname does not exist outside of greeter
- Then how helloTom() is possible?
- => closure: function that remembers its environment when it was declared



Closure

```
"use strict";

let fn;
{
  let cnt = 0;
  fn = () => { cnt += 1; return cnt; };
}

console.log(fn());
console.log(fn());
console.log(fn());

console.log(cnt);
```

```
1      script.js:9
2      script.js:10
3      script.js:11
✖ ▶ Uncaught ReferenceError: script.js:13
  a is not defined
    at script.js:13:13
```



Closure

```
"use strict";

function counter() {
  let n = 0;

  return {
    count: function() { return n++; },
    reset: function() { n = 0; },
  };
}

let cnt1 = counter();
let cnt2 = counter();

cnt1.count();
cnt2.count();
cnt1.count();
cnt2.reset();
```

- cnt1 and cnt2 are different counters
- What if cnt2 = Object.assign({}, cnt1); ?



IIFE

```
"use strict";  
  
let num = (function(a, b) {  
    return a + b;  
})(2, 3);  
  
console.log(num);
```

- IIFE (Immediately Invoked Function Expressions)
- Once the function is defined, it is immediately invoked



IIFE

```
"use strict";
```

```
let num = (function() {  
    let cnt = 0;  
    let idx = 0;  
    // do some complex job  
    return result;  
}) ();
```

```
// you can reuse these names
```

```
let cnt = 0;  
let idx = 0;  
console.log(num);
```

```
"use strict";
```

```
const num = (function() {  
    let n = 0;  
    return {  
        count: function() { return n++; },  
        reset: function() { n = 0; },  
    };  
}) ();
```

```
c.count();  
c.count();  
c.reset();  
c.count();
```